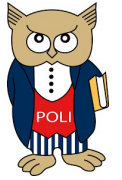




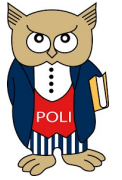
ESCUELA POLITÉCNICA NACIONAL



Sistemas Embebidos

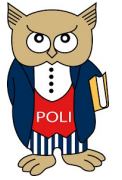
Semana 14

PhD. Germán E. Rodríguez



Integración y testeo de Hardware y Software de Sistemas Embebidos

- Ejemplos del mundo real de diseño de sistemas embebidos



Objetivo de la clase:

1. Explicar el funcionamiento del protocolo MQTT e implementar un sistema de comunicación entre un ESP32 y un servidor MQTT utilizando el modelo Publicador/Suscriptor



¿Qué es MQTT?

¿Por qué nace MQTT?

Hace algunos años la mayoría de dispositivos estaban conectados mediante cables.

Hoy existen millones de dispositivos:

- sensores
- relojes inteligentes
- vehículos
- drones
- cámaras
- electrodomésticos
- dispositivos médicos



¿Qué es MQTT?

Todos necesitan intercambiar información.

Sin embargo presentan limitaciones importantes:

- poca memoria
- poca capacidad de procesamiento
- poca batería
- redes lentas
- conexiones inestables

Los protocolos tradicionales como HTTP fueron diseñados para computadoras, no para dispositivos pequeños.

Aquí aparece MQTT.



¿Qué significa MQTT?

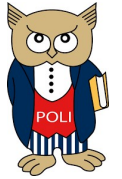
MQTT significa:

Message Queuing Telemetry Transport

Fue desarrollado en **1999** por IBM.

Su objetivo era comunicar sensores ubicados en oleoductos mediante enlaces satelitales muy lentos y costosos.

Con el tiempo se convirtió en el protocolo más utilizado en IoT.



¿Qué es MQTT?

MQTT es un protocolo de mensajería extremadamente ligero basado en el modelo:

Publicador — Broker — Suscriptor

No existe comunicación directa entre dispositivos.

Todo mensaje pasa por un servidor denominado **Broker**.



Analogía

Imaginemos un grupo de WhatsApp.

Una persona escribe: "Mañana no hay clases."

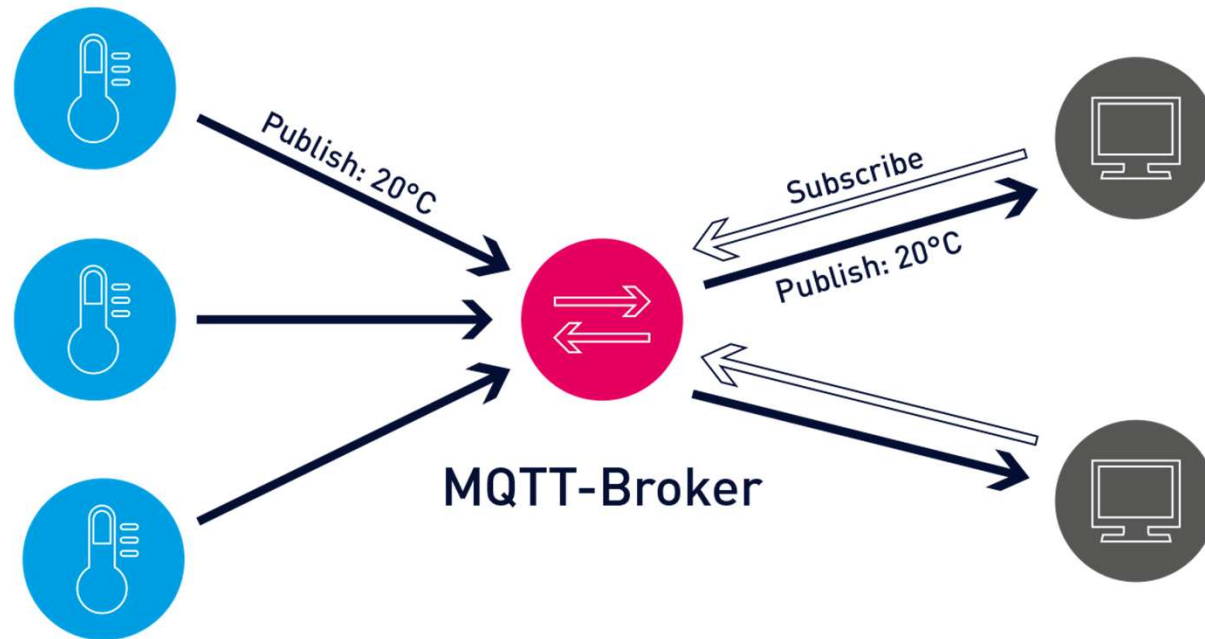
No envía el mensaje individualmente a cada estudiante.
Lo publica en un **grupo**.

Solo quienes pertenecen al **grupo** reciben el mensaje.

MQTT funciona exactamente igual.
El **grupo** de WhatsApp equivale al **Topic**.



Arquitectura MQTT





Componentes MQTT

1. Broker

Es el servidor MQTT.

Funciones:

- recibe mensajes
- almacena mensajes (opcional)
- administra conexiones
- envía mensajes
- controla usuarios
- administra Topics



Componentes MQTT

1. Broker

Ejemplos

- Mosquitto
- HiveMQ
- EMQX
- VerneMQ



EMQX





Componentes MQTT

2. Publisher (Publicador)

Es cualquier dispositivo que envía información.

Ejemplos

- ESP32
- Arduino
- Raspberry Pi
- Sensor de temperatura

Ejemplo:

Temperatura = 28°C

El ESP32 publica
casa/sala/temperatura

28



Componentes MQTT

3. Subscriber (Suscriptor)

Es quien desea recibir información.

Puede ser

- NodeRED
- Aplicación Android
- Dashboard Web
- Otro ESP32

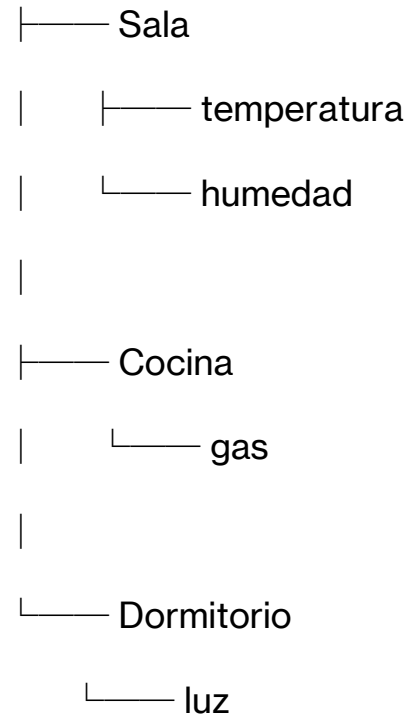


Componentes MQTT

Jerarquía de Topics

Se organizan como carpetas.

Casa





Wildcards

MQTT permite escuchar varios topics (+)

1. Escucha un solo nivel

casa/+/temperatura

Recibe

casa/sala/temperatura

casa/cocina/temperatura

2. Escucha todos los niveles

casa/#

Recibe todo.



Modelo Publicador/Suscriptor

- Una gran ventaja es que los dispositivos nunca se conocen entre sí.
- El ESP32 no sabe quién leerá el mensaje.
- Simplemente publica.
- Esto permite gran escalabilidad.



Comparación entre MQTT y HTTP

Característica	MQTT	HTTP
Comunicación	Publicador/Suscriptor	Cliente/Servidor
Consumo de ancho de banda	Muy bajo	Alto
Persistencia de conexión	Sí	No (por defecto)
Velocidad	Muy alta	Media
Ideal para IoT	Sí	No siempre
Encabezado	2 bytes (mínimo)	Decenas o cientos de bytes
Comunicación en tiempo real	Excelente	Limitada sin WebSockets



Ventajas de MQTT

- Muy ligero.
- Muy rápido.
- Bajo consumo energético.
- Ideal para microcontroladores.
- Funciona con conexiones lentas.
- Fácil de implementar.
- Escalable.
- Compatible con miles de dispositivos.



Desventajas de MQTT

- Requiere un Broker.
- No está pensado para transferir archivos grandes.
- La seguridad debe configurarse (TLS, autenticación).



MQTT en aplicaciones reales

- Hogares inteligentes.
- Agricultura de precisión.
- Ciudades inteligentes.
- Automatización industrial.
- Vehículos conectados.
- Domótica.
- Monitoreo ambiental.
- Hospitales.
- Monitoreo de pacientes.
- Redes de sensores inalámbricos.



¿Qué es Node-RED?

Node-RED es una plataforma de programación visual de código abierto que permite crear aplicaciones conectando bloques (llamados **nodos**) mediante un editor gráfico, sin necesidad de escribir grandes cantidades de código.

Fue desarrollado originalmente por **IBM** en 2013 y actualmente es un proyecto mantenido por la OpenJS Foundation.

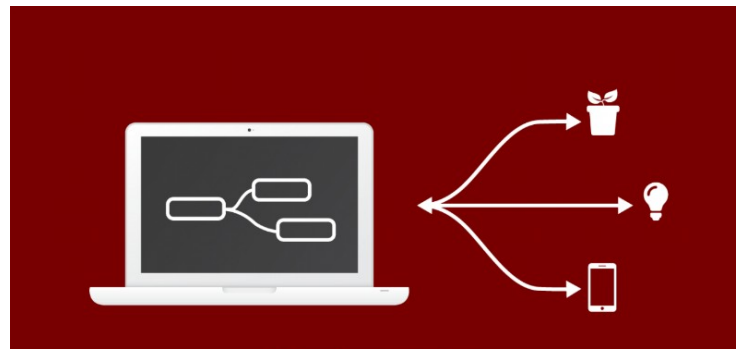
Su principal objetivo es facilitar la integración entre dispositivos IoT, servicios web, bases de datos, APIs y sistemas de automatización.



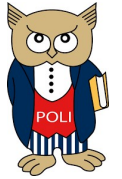
¿Qué es Node-RED?

Node-RED es una herramienta de programación basada en flujos (Flow-Based Programming) que permite conectar dispositivos, servicios y aplicaciones mediante una interfaz gráfica.

En lugar de programar línea por línea, el usuario arrastra nodos y los conecta para definir el flujo de información.



Node-RED

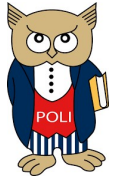


¿Por qué se llama Node-RED?

El nombre proviene de dos conceptos:

Node: cada bloque representa una función específica (leer un sensor, enviar un mensaje MQTT, realizar un cálculo, guardar datos, etc.).

RED: hace referencia al entorno gráfico donde esos nodos se conectan formando una red o flujo de trabajo.



¿Qué es un nodo?

Un **nodo** es un componente que ejecuta una función determinada.

Ejemplos:

- Leer MQTT
- Publicar MQTT
- Leer un puerto serial
- Consumir una API REST
- Enviar un correo electrónico
- Guardar datos en una base de datos
- Mostrar información en un Dashboard

Cada nodo tiene entradas y salidas que permiten conectarlo con otros nodos.



Tipos de nodos

Nodos de entrada (Input)

Obtienen información desde alguna fuente.

Ejemplos:

- MQTT In
- HTTP In
- Serial In
- Inject
- WebSocket



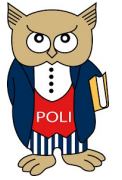
Tipos de nodos

Nodos de procesamiento

Transforman o procesan la información.

Ejemplos:

- Function
- Change
- Switch
- JSON
- Template



Tipos de nodos

Nodos de salida (Output)

Envían información a otro sistema.

Ejemplos:

- MQTT Out
- Debug
- HTTP Response
- Email
- Base de datos



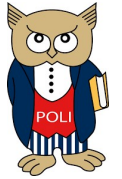
¿Cuándo usar Node-RED?

Node-RED es especialmente recomendable cuando se necesita:

- Crear prototipos IoT rápidamente.
- Integrar múltiples dispositivos y protocolos.
- Visualizar datos mediante paneles de control.
- Automatizar procesos sin desarrollar una aplicación completa.
- Enseñar conceptos de comunicaciones e integración de sistemas embebidos, ya que permite centrarse en la lógica del flujo más que en los detalles de programación de interfaces.



ESCUELA POLITÉCNICA NACIONAL



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Ejercicio Práctico usando Dockers y NodeRed



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Requisitos

- Docker Desktop
- ESP32 DevKit V1
- Arduino IDE
- Cable USB
- Red WiFi



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

1. Crear la carpeta del proyecto

Crear una carpeta en su equipo de trabajo.

`mqtt_lab`

Dentro de la carpeta crear la siguiente estructura:

`mqtt_lab`

`docker-compose.yml`

`mosquitto/
 config/
 data/
 log/`



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

2. Crear Docker Compose

Crear el siguiente archivo:

[docker-compose.yml](#)

version: '3'

services:

mqtt:

image: eclipse-mosquitto

container_name: mosquitto

restart: always

ports:

- "1883:1883"

volumes:

- ./mosquitto/config:/mosquitto/config
- ./mosquitto/data:/mosquitto/data
- ./mosquitto/log:/mosquitto/log

nodered:

image: nodered/node-red

container_name: nodered

restart: always

ports:

- "1880:1880"

volumes:

- ./data:/data

depends_on:

- mqtt



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

3. Configurar Mosquitto

Crear el siguiente archivo en el Directorio mosquitto

mosquitto/config/[mosquitto.conf](#)

Contenido

```
listener 1883
allow_anonymous true
persistence true
persistence_location /mosquitto/data/
```



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

4. Levantar Docker

Abrir consola/terminal
`docker compose up -d`

Verificar
`docker ps`

Debe aparecer:

`mosquitto`

`nodered`

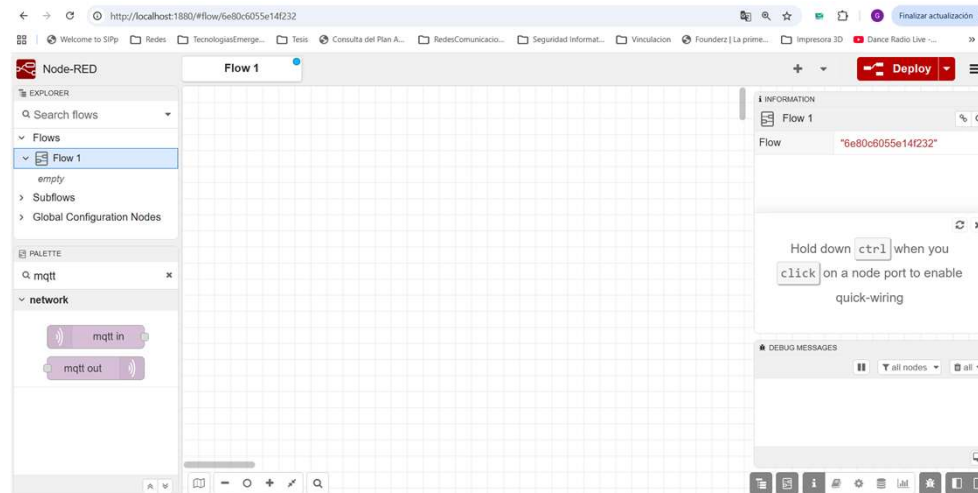


Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

5. Abrir Node-RED

Abrir navegador en la máquina anfitrión:

<http://localhost:1880>

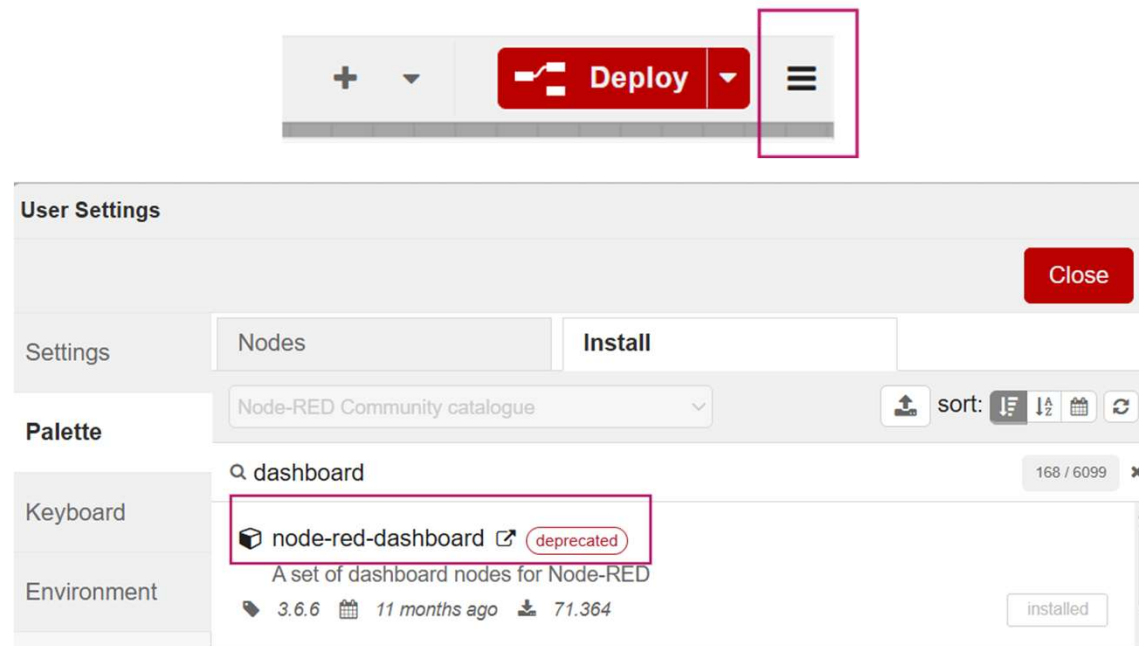




Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

6. Instalar Dashboard

En NodeRED, seleccionamos: Menu, luego Manage Palette, Install, Buscar y buscamos node-red-dashboard y al final: Instalar.





Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

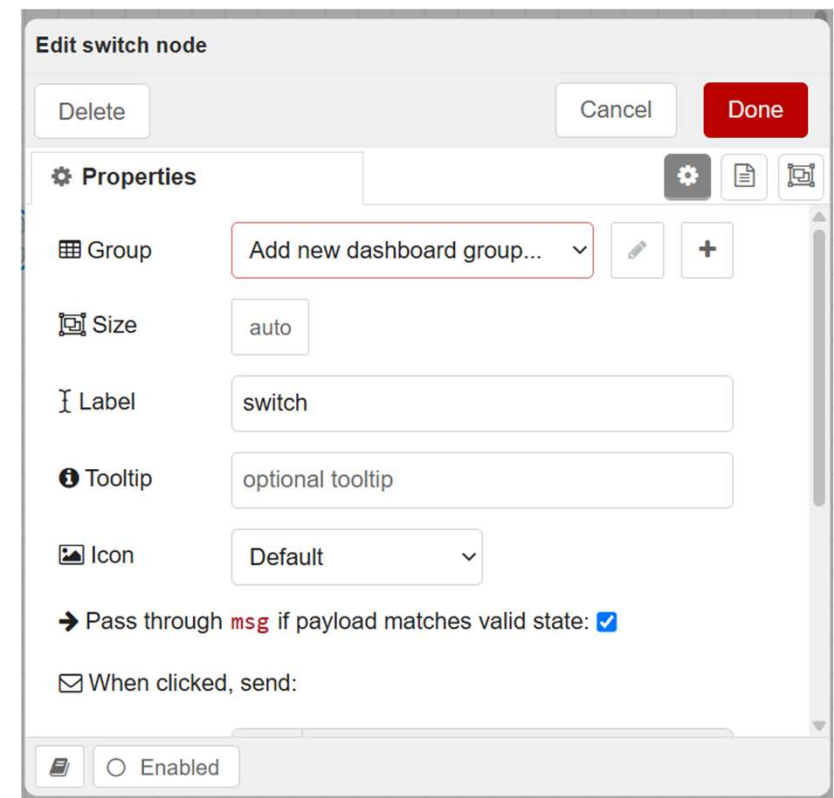
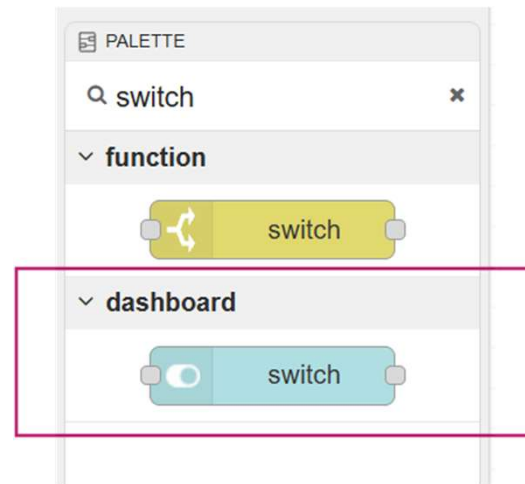
7. Crear Dashboard

Agregar
Dashboard: Switch

Configurar

Topic: esp32/led

Payload ON: ON
Payload OFF: OFF





Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Agregar nodo

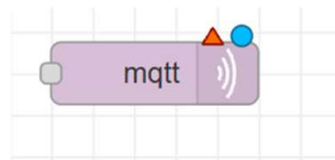
mqtt out

Servidor
mqtt
Puerto
1883
Topic
esp32/led

Conectar
Switch

mqtt out

Deploy.



Edit mqtt out node

Delete Cancel Done

⚙ Properties

🌐 Server none

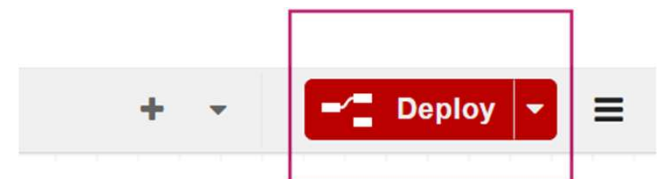
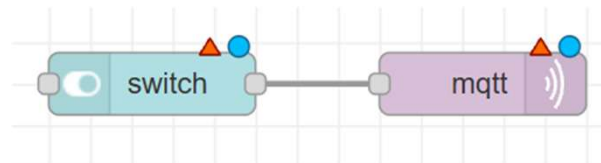
📄 Topic Topic

🔑 QoS Retain

🔖 Name Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

📄 Enabled



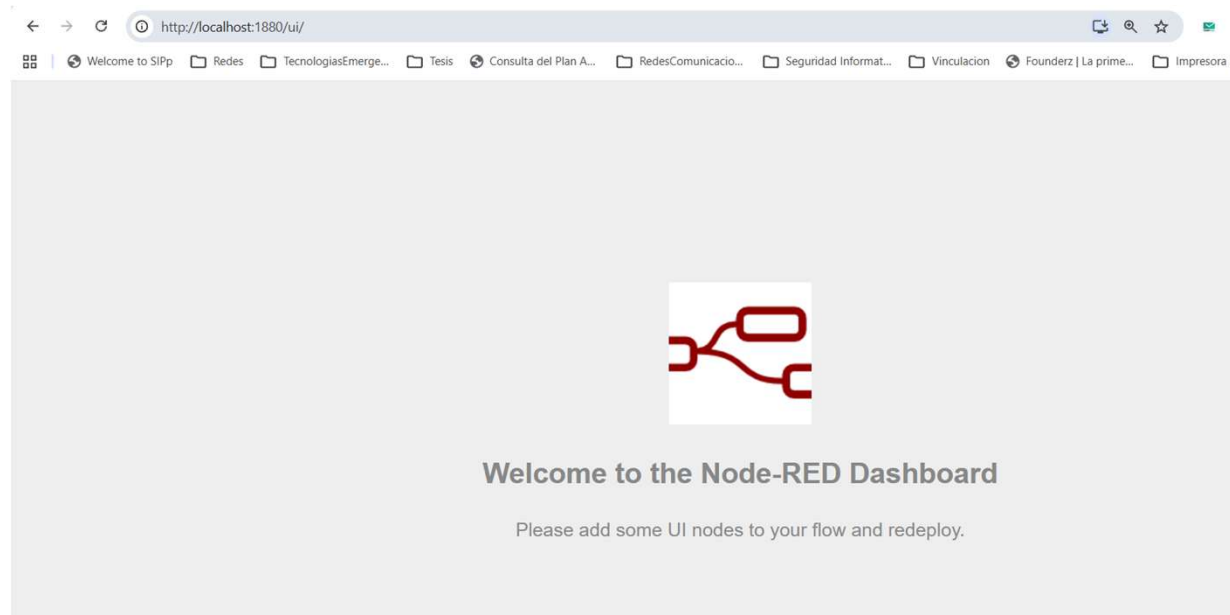


Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Dashboard

<http://localhost:1880/ui>

Todavía no hará nada porque falta el ESP32.





Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

8. Preparar Arduino IDE

Instalar librerías:

[PubSubClient](#) y [WiFi](#)

9. LED integrado

En muchos ESP32 DevKit el LED integrado está conectado al GPIO **2**.
Si su placa no enciende con GPIO 2, revisar el esquema específico del fabricante.



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

10. Código ESP32

```
#include <WiFi.h>
#include <PubSubClient.h>
```

```
const char* ssid = "SU_WIFI";
const char* password = "SU_PASSWORD";
```

```
// IP del computador donde corre Docker
const char* mqtt_server = "192.168.1.100";
```

```
WiFiClient espClient;
PubSubClient client(espClient);
```

```
#define LED 2
```

```
void setup_wifi()
{
    delay(10);

    WiFi.begin(ssid,password);

    while(WiFi.status()!=WL_CONNECTED)
    {
        delay(500);
    }
}

void callback(char* topic, byte* payload, unsigned int length)
{
    String mensaje="";

    for(int i=0;i<length;i++)
        mensaje+=(char)payload[i];

    if(mensaje=="ON")
        digitalWrite(LED,HIGH);

    if(mensaje=="OFF")
        digitalWrite(LED,LOW);
}
```



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

10. Código ESP32

```
void reconnect()
{
    while(!client.connected())
    {
        if(client.connect("ESP32"))
        {
            client.subscribe("esp32/led");
        }
        else
        {
            delay(2000);
        }
    }
}
```

```
void setup()
{
    pinMode(LED,OUTPUT);

    setup_wifi();

    client.setServer(mqtt_server,1883);

    client.setCallback(callback);
}

void loop()
{
    if(!client.connected())
        reconnect();

    client.loop();
}
```



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

11. Obtener IP del computador

En Windows: ipconfig

Buscar: IPv4 / 192.168.1.100

Esa IP se coloca en la variable: mqtt_server

No usar: localhost

porque el ESP32 está en otro dispositivo.

12. Subir programa

Seleccionar el board correspondiente: ESP32 Dev Module

Seleccionar el Puerto COM: COMx

Subir.



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

13. Probar

Abrir

<http://localhost:1880/ui>

Mover el Switch

OFF

↓

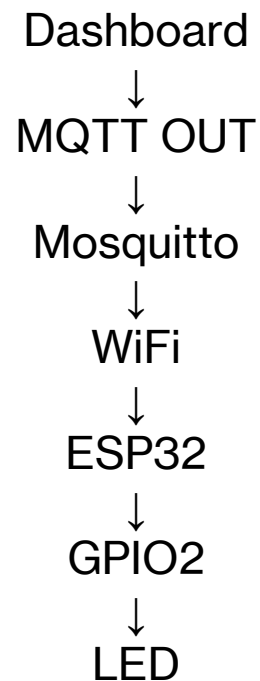
ON

El LED del ESP32 debe encenderse.



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Flujo de comunicación





Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Verificar mensajes MQTT

Desde el contenedor Mosquitto: `docker exec -it mosquitto sh`

Luego ejecutamos: `mosquitto_sub -h localhost -t esp32/led`

Cada vez que cambiemos el interruptor en el Dashboard, veremos:

ON
OFF
ON
OFF

Esto confirma que el broker está recibiendo los mensajes correctamente.



Capítulo 3: Integración y testeo de Hardware y Software de Sistemas Embebidos

Verificar mensajes MQTT

Desde el contenedor Mosquitto: `docker exec -it mosquitto sh`

Luego ejecutamos: `mosquitto_sub -h localhost -t esp32/led`

Cada vez que cambiemos el interruptor en el Dashboard, veremos:

ON
OFF
ON
OFF

Esto confirma que el broker está recibiendo los mensajes correctamente.